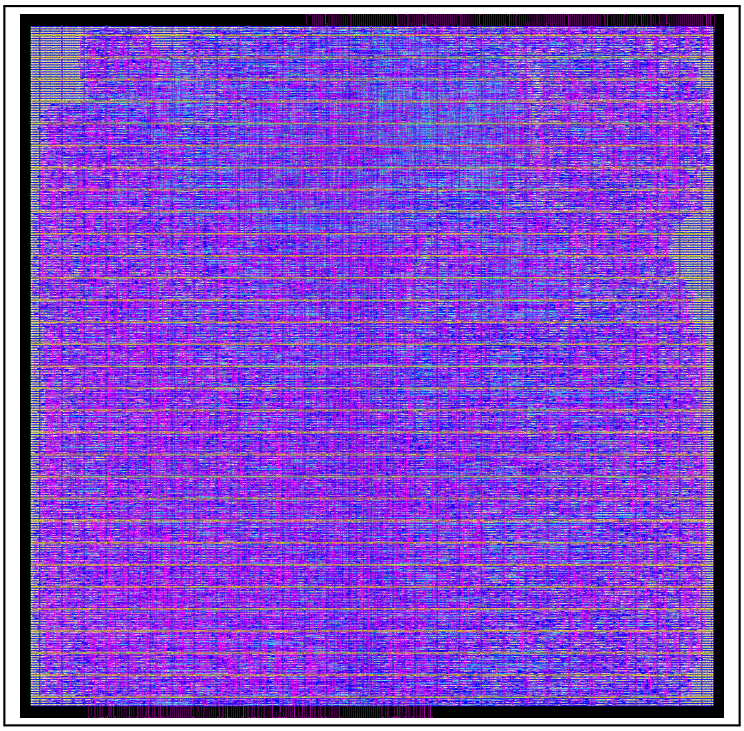
MICRON AI-AWARE CACHE & PREFETCH ENGINE
PHYSICAL DESIGN REPORT

From Verilog RTL to a Placed-and-Routed SkyWater 130 nm Layout with Open-Source Tools (Yosys, OpenROAD, KLayout)



The routed chip: 798 um x 798 um, 27,295 logic cells, 0 routing DRC violations

Event	Micron hackathon - AI-Aware Cache & Prefetch Engine RTL Challenge
Team / Author	Team Agrimos - Kotapati Kishore (Roll No. EE26E016)
Affiliation	B.Tech EEE, NIT Manipur; final year at IIT Madras (Dept. of Electrical Engineering, student exchange)
Date	1 November 2026
Scope	Physical implementation of the cache + prefetch engine RTL, 512 B cache configuration (same RTL, SETS = 8). Companion to the main Technical Report.
Headline result	798 x 798 um die, 0.376 mm2 cell area, 0 DRC violations after routing, about 69 MHz (typical corner), about 103 mW estimated
Source code	https://github.com/Kishore-Kotapati/micron_cache_prefetch (folder physical/)

Table of Contents

Executive Summary **5**

1. Introduction **5**

1.1 Why go beyond RTL . . . 5

1.2 What this report covers . . . 5

1.3 Relation to the main report . . . 6

2. Background: How a Chip Layout Is Made **6**

2.1 Key ideas . . . 6

3. Design Under Implementation **7**

3.1 What is on the chip . . . 7

3.2 The 512 B configuration . . . 7

4. Functional Check of the 512 B Configuration **8**

5. Technology and Tools **8**

5.1 SkyWater 130 nm . . . 8

5.2 Tools and versions . . . 8

5.3 Corner and constraints . . . 9

6. Logic Synthesis **9**

7. Floorplan and Power Grid **10**

8. Placement **11**

9. Clock Tree Synthesis **12**

10. Routing **13**

10.1 Global routing . . . 13

10.2 Detailed routing . . . 13

11. Timing, Power and Electrical Checks **14**

11.1 Setup timing . . . 14

11.2 Hold timing . . . 14

11.3 Power . . . 14

11.4 Electrical rule checks . . . 15

11.5 Summary of checks . . . 15

12. Layout Pictures **15**

13. Challenges, Fixes and Limitations **20**

13.1 Problems met and how they were solved . . . 20

13.2 Limitations (stated honestly) . . . 20

13.3 Future work . . . 21

14. Conclusion **21**

References **21**

Appendix A - Files of the Physical Design **22**

Appendix B - Commands to Reproduce **22**

List of Figures

Figure 1: The open-source ASIC flow used in this project 6

Figure 2: Flip-flops per design block (colour graph) 9

Figure 3: Floorplan: die, core, standard-cell rows and the power-strap grid (drawn to scale) 10

Figure 4: Where each block was placed: every dot is a flip-flop coloured by its RTL block; grey dots are logic gates (colour graph) 12

Figure 5: DRC violations after each detailed-routing iteration (colour graph, log scale) 13

Figure 6: Wire length per layer after detailed routing and global-route track usage (colour graph) 13

Figure 7: Whole routed chip, all layers (798 um x 798 um) 16

Figure 8: Metal layers only (met1-met5 and vias): the wiring of the chip 17

Figure 9: 40 um x 40 um window at the centre: rows of cells with routed wires on top 18

Figure 10: 12 um x 12 um close-up: transistor diffusion (green), polysilicon gates (red), local interconnect and metal wires (blue/purple) 19

Figure 11: One-page overview of the physical design with its measured numbers 20

List of Tables

Table 1: Key results of the physical implementation 5

Table 2: Flow steps 6

Table 3: Blocks placed on the chip 7

Table 4: Cache size parameters (computed from the RTL equations) 7

Table 5: Icarus Verilog results for the 512 B configuration (physical/sim_512B_results.txt) 8

Table 6: Tools used (all free and open-source) 8

Table 7: Most used cells after synthesis (physical/out/synth_stat.txt) 9

Table 8: Flip-flops traced to RTL blocks 10

Table 9: Floorplan and power-distribution settings 11

Table 10: Placement and repair statistics 11

Table 11: Clock-tree statistics 12

Table 12: Global-routing congestion report (31,551 nets, estimated length 1,741 mm) 13

Table 13: Detailed-routing results 14

Table 14: Critical path broken into parts (from out/final_reports.txt) 14

Table 15: Power estimate at 100 MHz, typical corner (report_power) 14

Table 16: Status of physical checks 15

Table 17: Problems and fixes during physical design 20

Table 18: Physical-design files in the repository 22

List of Abbreviations

ASIC	Application-Specific Integrated Circuit
CTS	Clock Tree Synthesis
DEF	Design Exchange Format (placed / routed layout as text)
DRC	Design Rule Check
GDS / GDSII	Graphic Data System - the final mask-layout file sent to a foundry
HPWL	Half-Perimeter Wire Length (placement wire estimate)
LEF	Library Exchange Format (cell outlines and pins)
LVS	Layout Versus Schematic
MSHR	Miss Status Holding Register
PDK	Process Design Kit
PDN	Power Distribution Network
RTL	Register-Transfer Level
SDC	Synopsys Design Constraints (timing constraints format)
SPEF	Standard Parasitic Exchange Format
SRAM	Static Random-Access Memory
STA	Static Timing Analysis
TNS / WNS	Total / Worst Negative Slack

Units and Notation

um, mm2	micrometre; square millimetre (1 mm2 = 1,000,000 um2)
ns, MHz	nanosecond; megahertz (f = 1000 / T, T in ns)
T_target	target clock period = 10 ns (100 MHz)
slack	time to spare on a path: required time - arrival time (negative = too slow)
met1 ... met5, li1	SkyWater metal layers (li1 = local interconnect, lowest)

Executive Summary

The main Technical Report showed that the Agrimos cache and prefetch engine works and is fast, using RTL simulation. This report answers the next question: **can the same Verilog become a real chip layout?** The RTL was taken through a complete open-source ASIC flow on the SkyWater 130 nm process: synthesis to real standard cells, floorplan, power grid, placement, clock tree, routing, and finally a GDS layout file and pictures.

A cache built only from flip-flops grows very quickly, and a 4 KB array made this way is too large for a reasonable run of the free flow on the available machine. The layout therefore uses **the same RTL with one parameter changed**: SETS = 8 instead of 64, giving a 512 B, 4-way cache with the complete prefetch engine. This configuration was simulated again on all nine workloads with the normal self-checking testbench, and every check passed.

Table 1: Key results of the physical implementation

Measure	Result	Source
Logic cells after synthesis	27,295 (6,103 flip-flops)	Yosys stat
Die size	797.6 um x 797.6 um (0.636 mm2)	OpenROAD floorplan
Placed cell area / core utilisation	0.376 mm2 / 63 %	report_design_area
Routing DRC violations	0 (after 18 TritonRoute iterations)	TritonRoute
Worst setup slack @ 10 ns	-4.39 ns -> about 14.4 ns, about 69 MHz	OpenROAD STA
Hold violations	none found	repair_timing
Estimated power	103 mW (default switching activity)	report_power
Functional check of 512 B version	18/18 runs pass; CNN 2.78x, LLM 2.13x, RecSys 2.05x speedup	Icarus Verilog

The 100 MHz target was **not** met. The slowest path is a 32-bit subtraction inside the stride predictor that synthesis built as a ripple-carry chain (23 carry cells in a row) when it computes the distance between the new address and a stream's last address. Faster adder logic or one extra pipeline stage in the predictor is the clear next step. These results are not a full sign-off: only the typical corner was analysed, wire parasitics were estimated rather than extracted, and Magic DRC, LVS and antenna checks were not run. Section 13 lists all the limitations.

1. Introduction

1.1 Why go beyond RTL

RTL simulation proves that a design does the right thing. It does not show how big the design is, how fast it can be clocked or how much power it uses. Those answers come from **physical design**: turning the logic into real transistors and real metal wires on a real manufacturing process. For a hardware challenge from a memory company, showing that the design can be laid out is a strong extra piece of evidence.

1.2 What this report covers

- Section 2 explains the ASIC flow in simple words.
- Sections 3-4 describe the design that was laid out and how its correctness was re-checked.
- Sections 5-10 go step by step: technology and tools, synthesis, floorplan and power, placement, clock tree, routing.
- Section 11 reports timing, power and electrical checks, and Section 12 shows the layout pictures.
- Sections 13-15 cover problems met, limitations and conclusions.

1.3 Relation to the main report

All performance claims of the project (2.26x geometric-mean speedup, etc.) are for the 4 KB design and come from the main Technical Report. This report adds the physical view and uses only the 512 B configuration. No number in this report is estimated by hand: each one is read from the Yosys or OpenROAD logs by the script that builds this PDF.

2. Background: How a Chip Layout Is Made

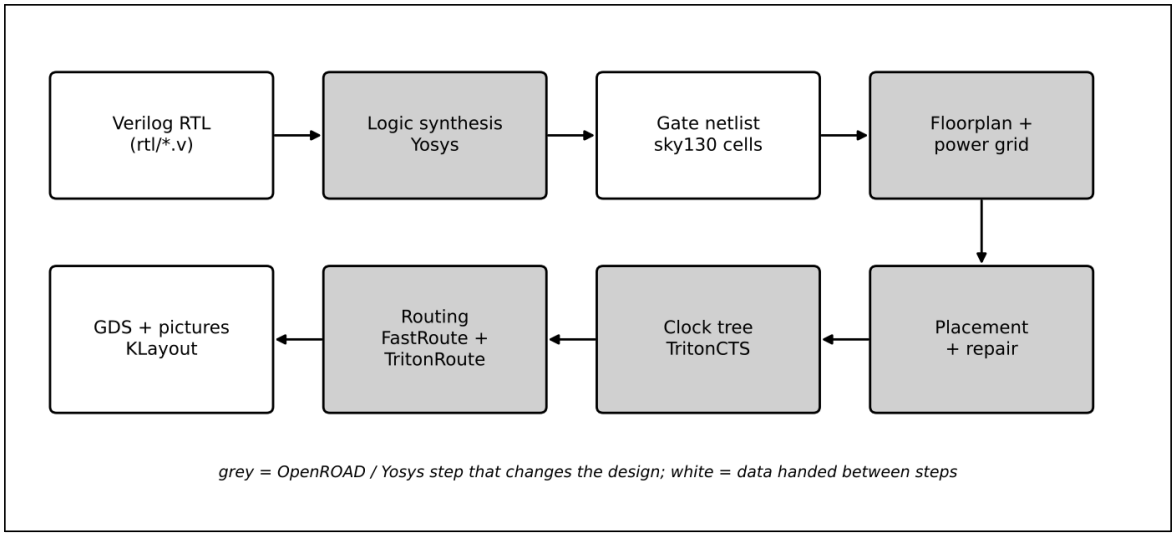


Figure 1: The open-source ASIC flow used in this project

Table 2: Flow steps

Step	What it does (simple words)	Tool used
Logic synthesis	Turns Verilog into a list of real library cells (AND, OR, MUX, flip-flops...) and how they connect	Yosys + ABC
Floorplan	Chooses the chip size, makes rows where cells sit, adds well-tap and end-cap cells	OpenROAD (ifp, tapcell)
Power grid	Draws the VDD/GND metal grid that feeds every cell	OpenROAD (pdngen)
Pin placement	Puts the input/output pins around the edge of the die	OpenROAD (ppl)
Placement	Finds a position for every cell so wires are short, then fixes overlaps	RePlAce + OpenDP
Repair	Adds buffers and resizes cells to fix slow or over-loaded signals	OpenROAD Resizer
Clock tree	Builds a balanced tree of buffers so the clock reaches all flip-flops at nearly the same time	TritonCTS
Global route	Plans which region and layer each wire uses	FastRoute
Detailed route	Draws the exact metal shapes and vias, following all spacing rules	TritonRoute
Analysis	Checks timing (STA), power, and electrical limits	OpenSTA inside OpenROAD
Stream-out	Merges routed wires with transistor-level cell drawings into GDS	KLayout

2.1 Key ideas

- **Standard cells** are small pre-drawn circuits of equal height that sit in rows, like bricks in a wall.
- **Utilisation** is the fraction of the core area covered by cells; the rest is space for routing and filler.
- **Slack** is how much time a signal has to spare before the next clock edge:

$$slack = required\ time - arrival\ time;\ f_max \sim 1 / (T_target - WNS)$$

- **DRC** (design rule check) violations are wires that are too close, too thin or overlapping. A layout must have zero.
- **GDS** is the final file. It contains every polygon of every layer and is what a foundry would manufacture.

3. Design Under Implementation

3.1 What is on the chip

The layout contains the complete design from the main report minus the DRAM model, which is a simulation model only. The wrapper *layout_top.v* connects the cache controller and the prefetch engine exactly as in *synth/synth_top.v*; the DRAM interface becomes chip pins, where a real memory controller would connect.

Table 3: Blocks placed on the chip

Block	Function
cache_controller	hit/miss check, demand FSM, 8 MSHRs, prefetch issue, counters
tag_array	tags, valid bits, prefetched bits for every line
cache_memory	the data lines (16 B each)
replacement_policy	true-LRU ages (2 bits per way)
stride_predictor	4 stream entries: last address, stride, confidence
prefetch_engine	candidate queue, duplicate filter, adaptive degree, accuracy monitor

3.2 The 512 B configuration

Only one parameter changes. All address fields are derived from it by the same RTL equations:

index bits = log2(SETS); tag bits = 32 - 4 (offset) - index bits

Table 4: Cache size parameters (computed from the RTL equations)

Parameter	Main design (4 KB)	Layout design (512 B)
SETS x WAYS x LINE	64 x 4 x 16 B	8 x 4 x 16 B
Index / tag bits	6 / 22	3 / 25
Data bits (= data flip-flops)	64 x 4 x 128 = 32,768	8 x 4 x 128 = 4,096
Tag + valid + prefetched bits	256 x 24 = 6,144	32 x 27 = 864
LRU age bits	256 x 2 = 512	32 x 2 = 64
Prefetch engine, MSHRs, FSM	unchanged	unchanged

The flip-flop counts predicted in Table 4 for the 512 B design (4,096 data, 864 tag/valid/prefetched, 64 LRU) are exactly the counts found later in the synthesised netlist (Section 6), which confirms that the parameter change worked as intended.

4. Functional Check of the 512 B Configuration

Before any layout work, the 512 B version was simulated with the project's normal testbench. The only edits are the parameter on the instance and the matching testbench constants (*physical/tb_layout_cfg.v*). Each run executes all five self-checks of the main report (correct data, counters consistent, no duplicate lines, prefetch accounting, MSHRs drain).

Table 5: Icarus Verilog results for the 512 B configuration (physical/sim_512B_results.txt)

Trace	Baseline cycles	Final design cycles	Speedup	Self-checks
sequential	112,571	29,172	3.86x	2/2 passed
stride	133,557	25,688	5.20x	2/2 passed
random	178,139	178,139	1.00x	2/2 passed
repeated	132,485	132,239	1.00x	2/2 passed
mixed	182,305	108,827	1.68x	2/2 passed
capacity	84,464	22,016	3.84x	2/2 passed
cnn	193,032	69,393	2.78x	2/2 passed
llm	82,323	38,686	2.13x	2/2 passed
recsys	124,415	60,817	2.05x	2/2 passed

All 18 runs pass. The prefetcher still helps strongly on regular streams and AI-style traces, and it never slows the random or repeated traces down. Speedups differ from the 4 KB numbers because the smaller cache misses more often.

5. Technology and Tools

5.1 SkyWater 130 nm

SKY130 is an open-source process design kit released by SkyWater Technology and Google. It is a real, manufacturable 130 nm CMOS process. The *sky130_fd_sc_hd* (high density) standard-cell library was used. Its cells sit on a 0.46 um x 2.72 um site grid, and it routes on one local-interconnect layer (li1) and five metal layers (met1-met5).

5.2 Tools and versions

Table 6: Tools used (all free and open-source)

Tool	Version	Role	How obtained
Yosys	0.33	logic synthesis + ABC technology mapping	Ubuntu 24.04 package
OpenROAD	2.0 build f12e2f47 (2022)	floorplan, PDN, placement, CTS, routing, STA, power	litex-hub conda channel
sky130A PDK	open_pdks 1.0.471	liberty, LEF, cell GDS, KLayout layer files	litex-hub conda channel
KLayout	0.30.12	GDS stream-out and layout pictures	pip install klayout
Python + Matplotlib	3.13	block map, report figures	system
Icarus Verilog	12.0	functional check of the 512 B version	Ubuntu package

No commercial EDA tool was used. The OpenROAD build is older than the newest release; it was chosen because its dependencies install cleanly from the conda-forge channel in the available environment.

5.3 Corner and constraints

```
create_clock -name clk -period 10.0 [get_ports clk]        ;# 100 MHz target
set_input_delay  2.0 -clock clk [all inputs except clk]
set_output_delay 2.0 -clock clk [all_outputs]
set_load 0.02 [all_outputs]
Library corner: sky130_fd_sc_hd__tt_025C_1v80 (typical process, 25 C, 1.80 V)
```

6. Logic Synthesis

Yosys reads the six RTL modules and the wrapper, flattens the hierarchy, maps flip-flops with *dfflibmap* and the combinational logic with ABC onto sky130 cells, ties constants to *conb* cells and writes a gate-level netlist.

```
read_liberty -lib sky130_fd_sc_hd__tt_025C_1v80.lib
read_verilog -sv ../rtl/*.v layout_top.v ; hierarchy -top layout_top
synth -top layout_top -flatten
dfflibmap -liberty $lib ; abc -D 10000 -liberty $lib
hilomap ... ; setundef -zero ; splitnets ; insbuf -buf sky130_fd_sc_hd__buf_2 A X
check -assert ; write_verilog out/layout_top.netlist.v
```

Table 7: Most used cells after synthesis (physical/out/synth_stat.txt)

Cell type	Count	Meaning
dfxtp_1	6,103	D flip-flop (one stored bit)
nand2_1	4,898	2-input NAND
mux2_1	3,816	2:1 multiplexer
a21oi_1	2,619	AND-OR-invert
nor2_1	1,959	2-input NOR
o21ai_0	1,887	OR-AND-invert
mux2i_1	1,213	2:1 inverting multiplexer
mux4_2	611	4:1 multiplexer
xnor2_1	386	2-input XNOR
lpflow_isobufsrc_1	273	isolation buffer (used as AND-NOT logic)
clkinv_1	267	inverter
nand2b_1	254	2-input NAND
(all 70 types)	27,295	total cell area 0.269 mm2 before placement

Flip-flops and multiplexers dominate. This is typical of storage built from registers: every stored bit is a flip-flop, and reading one line out of 32 needs wide multiplexer trees. Each flip-flop can be traced back to its RTL block through the name of the register it drives:

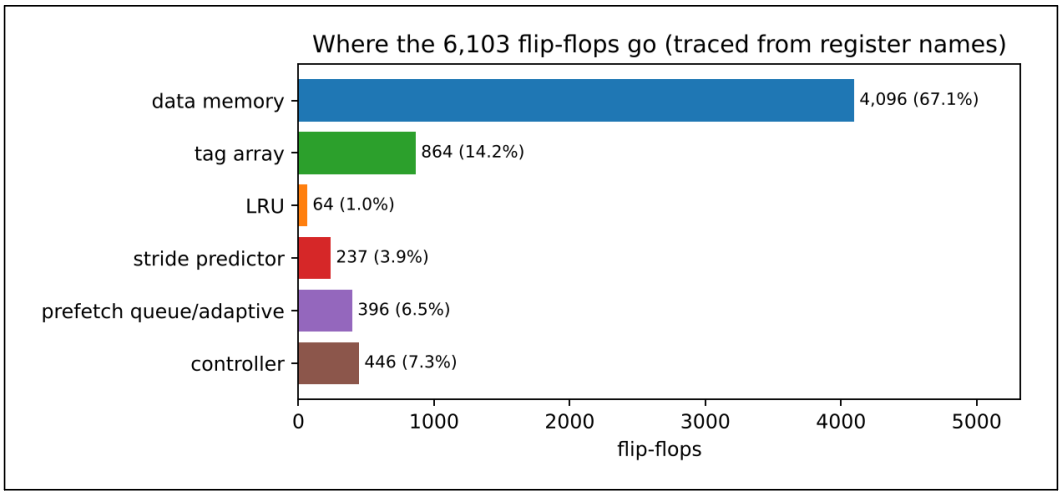


Figure 2: Flip-flops per design block (colour graph)

Table 8: Flip-flops traced to RTL blocks

Block	Flip-flops	Share
Cache data memory (8 sets x 4 ways x 16 B)	4,096	67.1 %
Tag array + valid + prefetched bits	864	14.2 %
LRU replacement ages	64	1.0 %
Stride predictor (4 streams)	237	3.9 %
Prefetch queue + adaptive-degree logic	396	6.5 %
Cache controller (FSM, 8 MSHRs, counters, outputs)	446	7.3 %
Total	6,103	100 %

Two thirds of all flip-flops are the cache data array, and the whole prefetch engine (predictor plus queue and adaptive logic) needs 633 flip-flops, about 10 % of the total. So, as in the main report's Yosys analysis, the intelligence is cheap compared with the storage it serves.

7. Floorplan and Power Grid

The floorplan was created for a target utilisation of 45 % with a square aspect ratio and a 12 um margin between core and die edge. After repair buffers and tap cells were added, the final utilisation reached the value in Table 1.

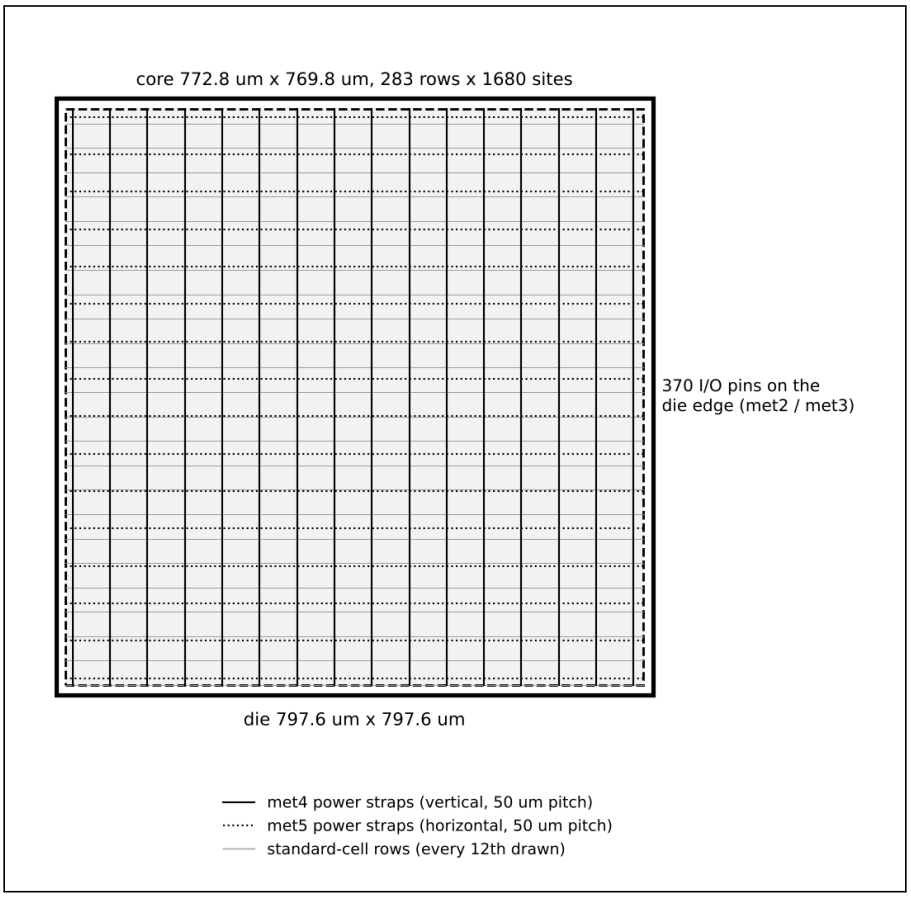


Figure 3: Floorplan: die, core, standard-cell rows and the power-strap grid (drawn to scale)

Table 9: Floorplan and power-distribution settings

Item	Value
Die	797.6 um x 797.6 um
Core	772.8 um x 769.8 um
Standard-cell rows	283 rows x 1680 sites (site 0.46 um x 2.72 um)
Well-tap cells / end-caps	7,837 tapvpwrvgnd_1 every 14 um / 566 decap_3
Power rails	met1, 0.48 um wide, along every cell row (VPWR / VGND)
Power straps	met4 vertical and met5 horizontal, 1.6 um wide, 50 um pitch, via stacks met1-met4-met5
I/O pins	370 pins on the die edge, horizontal on met3, vertical on met2

8. Placement

Global placement (RePlAce) spreads the cells like charged particles until they no longer overlap too much, while keeping connected cells close. The Resizer then repairs electrical problems and the detailed placer snaps every cell to a legal site.

Table 10: Placement and repair statistics

Measure	Value
Slew violations found before repair	6,776
Capacitance violations found	48
Buffers inserted / cells resized	549 / 1,901
Wire estimate after legalisation (HPWL)	734 mm
HPWL after clock tree and timing repair	844 mm

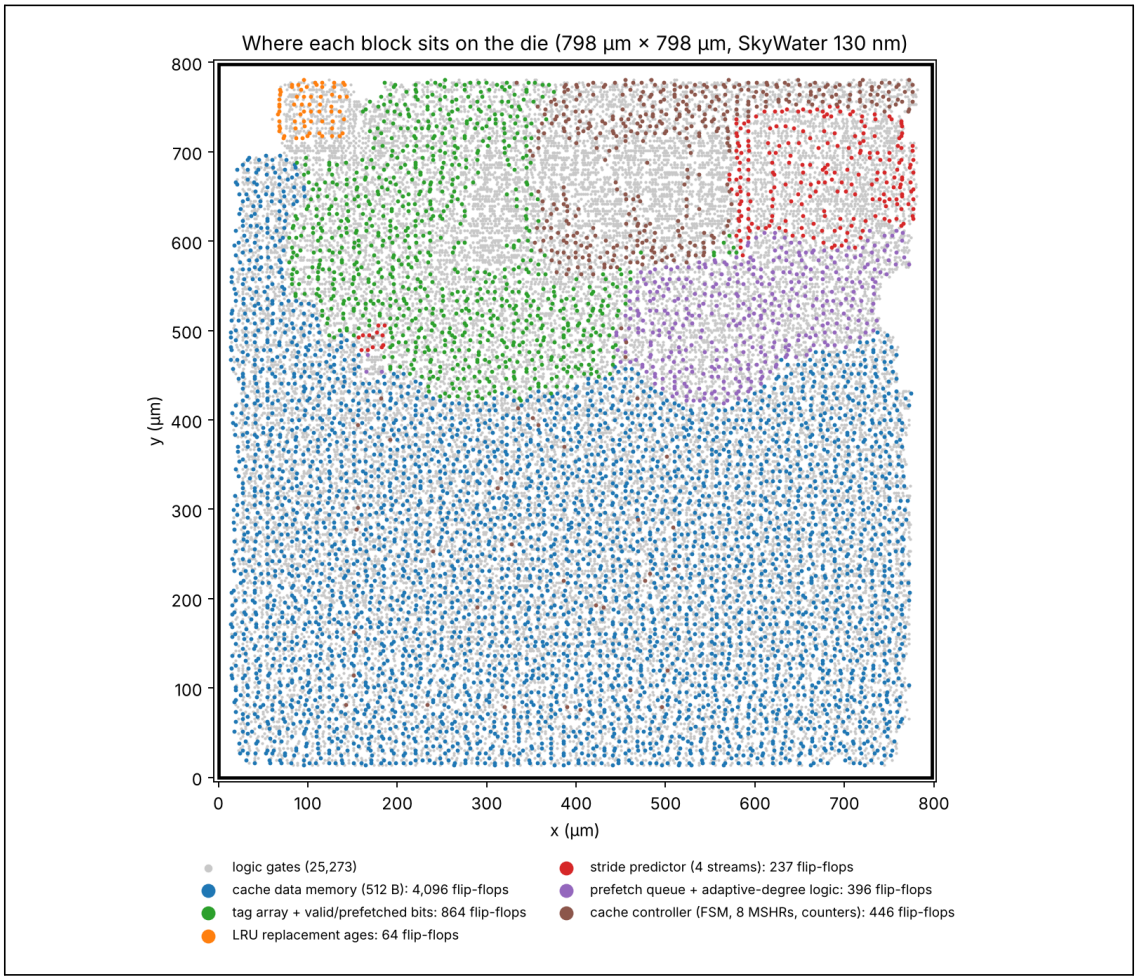


Figure 4: Where each block was placed: every dot is a flip-flop coloured by its RTL block; grey dots are logic gates (colour graph)

The placer was not told where any block should go, yet each block ended up in its own region. The data memory fills the lower half; the tag array sits above it, next to the controller; the stride predictor and the prefetch queue share the upper-right corner, close to each other because they exchange many signals. This grouping is a good sign that the design has clean, local connections inside each block.

9. Clock Tree Synthesis

Every flip-flop must see the clock edge at almost the same moment. TritonCTS builds a tree of clock buffers from the clock pin to all sinks; afterwards the timing is recomputed with the real (propagated) clock delays and repaired again.

Table 11: Clock-tree statistics

Measure	Value
Clock sinks (flip-flops)	6,103
Leaf buffers	2,937
Buffer levels (path depth)	11 - 13
Average sink wire length	1,533 um
Root / tree buffers	clkbuf_16 / clkbuf_4, clkbuf_8
Buffers added by post-CTS timing repair	77
Hold violations after CTS	none found

10. Routing

10.1 Global routing

Global routing divides the chip into tiles and plans the path and layer of every net. Overflow (more wires than tracks in a tile) must be zero before detailed routing.

Table 12: Global-routing congestion report (31,551 nets, estimated length 1,741 mm)

Layer	Track resource	Demand	Usage	Overflow
li1	0	0	0.00 %	0
met1	129,287	68,435	52.93 %	0
met2	132,468	72,417	54.67 %	0
met3	131,556	15,321	11.65 %	0
met4	75,870	13,704	18.06 %	0
met5	26,220	0	0.00 %	0

10.2 Detailed routing

TritonRoute draws the exact metal shapes. It starts with many conflicts and repeats rip-up-and-reroute passes until the layout is clean. Here it needed 18 passes and 00:40:47 (h:mm:ss) on two CPU cores.

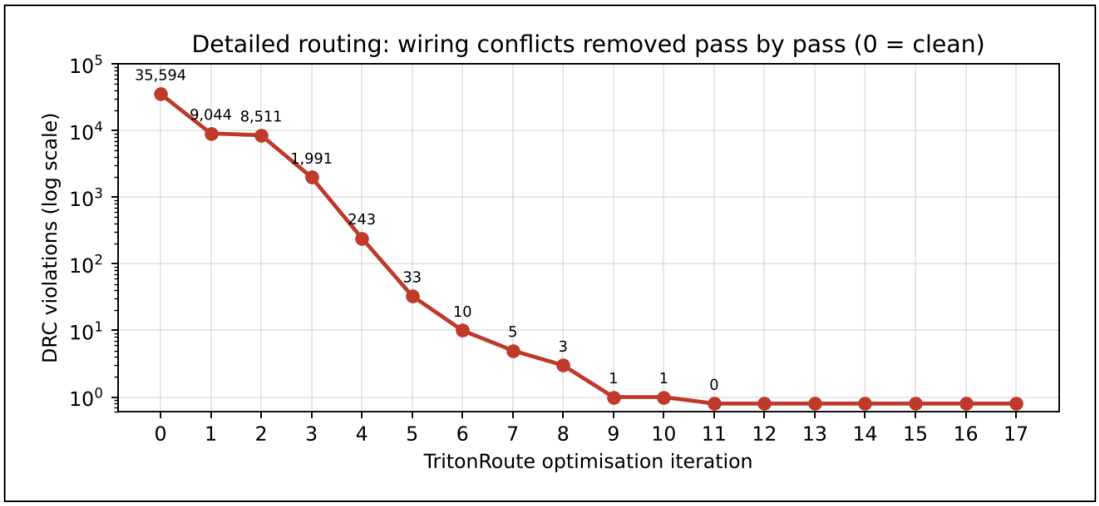


Figure 5: DRC violations after each detailed-routing iteration (colour graph, log scale)

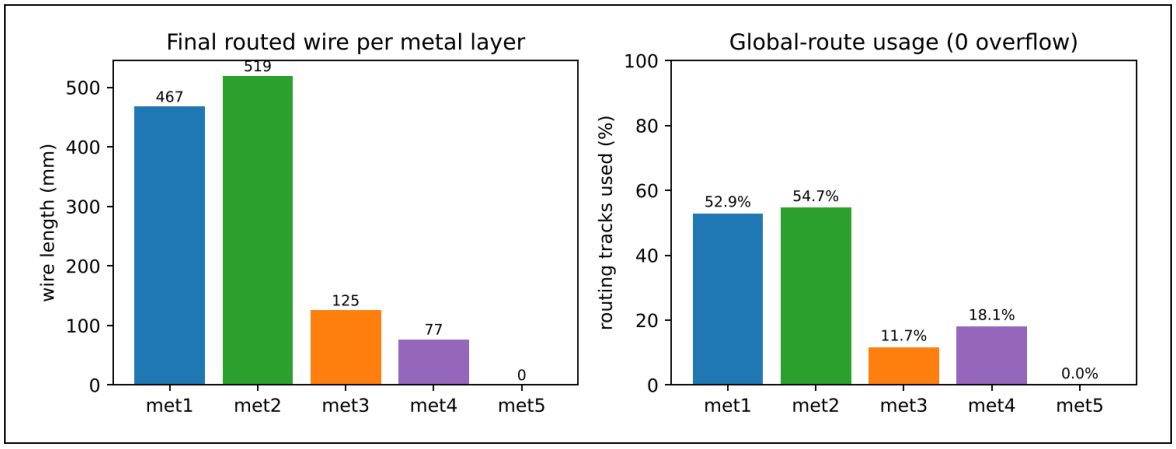


Figure 6: Wire length per layer after detailed routing and global-route track usage (colour graph)

Table 13: Detailed-routing results

Measure	Value
Final DRC violations (TritonRoute)	0
Total routed wire	1,188 mm (about 1.19 m)
Vias	241,277
Most used layers	met2 (vertical) and met1 (horizontal)
Unused layer	met5 carries only power straps; no signal needed it

11. Timing, Power and Electrical Checks

11.1 Setup timing

Static timing analysis with propagated clocks gives a worst negative slack of **-4.39 ns** against the 10 ns target (total negative slack -1,080.0 ns). The design therefore works at a clock period of about 14.4 ns:

$$f_max \sim 1000 / (10.0 - (-4.39)) = 1000 / 14.39 \sim 69\ MHz$$

Table 14: Critical path broken into parts (from out/final_reports.txt)

Part of the path	Cells	Delay (ns)
Clock arrival at launching flip-flop (u_pred.tr_addr[0])	clock tree	2.04
Flip-flop clock-to-Q	1	0.00
32-bit subtraction d = new address - stream's last address (stride / window check), ripple carry	32 cells (23 maj3)	10.37
Compare, select and degree/trigger logic	18	3.73
Data arrival at u_pf.trig_step[14]		16.14
Required time (10 ns + clock arrival - setup)		11.75

More than half of the path is one carry chain. ABC built the 32-bit subtractor in the stride predictor as a ripple-carry adder because that uses the least area. Two simple fixes are possible: (1) map that subtractor to a faster carry-lookahead structure, or (2) register the stride in one cycle and compare it in the next. Neither changes what the prefetcher predicts; option (2) would only delay training by one cycle.

11.2 Hold timing

The timing repair reported no hold violations after clock-tree synthesis.

11.3 Power

Table 15: Power estimate at 100 MHz, typical corner (report_power)

Group	Internal (mW)	Switching (mW)	Leakage (nW)	Total (mW)
Sequential	26.5	1.3	52	27.8
Combinational	55.0	20.2	173	75.1
Total	81.5	21.5	225	103.0

The estimate uses OpenROAD's default switching activity, not activity from a real workload, so it should be read as an order of magnitude. Leakage is tiny in this 130 nm process; most power is internal cell power from the many flip-flops and multiplexers clocked every cycle.

11.4 Electrical rule checks

127 small electrical warnings remain: 126 pins whose signal transition is 1.51-1.52 ns against a 1.50-1.51 ns limit, and one output whose load is 0.55 pF at the 0.55 pF limit. They are 1-2 % over the limit and could be removed with one more buffering pass, but they are reported here as measured.

11.5 Summary of checks

Table 16: Status of physical checks

Check	Status
Routing DRC (TritonRoute rule deck)	PASS - 0 violations
Placement legality (check_placement)	PASS
Global-route overflow	PASS - 0
Setup timing @ 100 MHz	FAIL - WNS -4.39 ns (passes at about 69 MHz)
Hold timing	PASS - none found
Max slew / max capacitance	127 marginal warnings
Magic / KLayout full-deck DRC, LVS, antenna	NOT RUN
Gate-level simulation of the routed netlist	NOT RUN

12. Layout Pictures

All pictures are rendered by KLayout directly from the routed DEF merged with the SkyWater cell GDS, using the official SKY130 layer colours. They are screenshots of the real layout, not drawings.

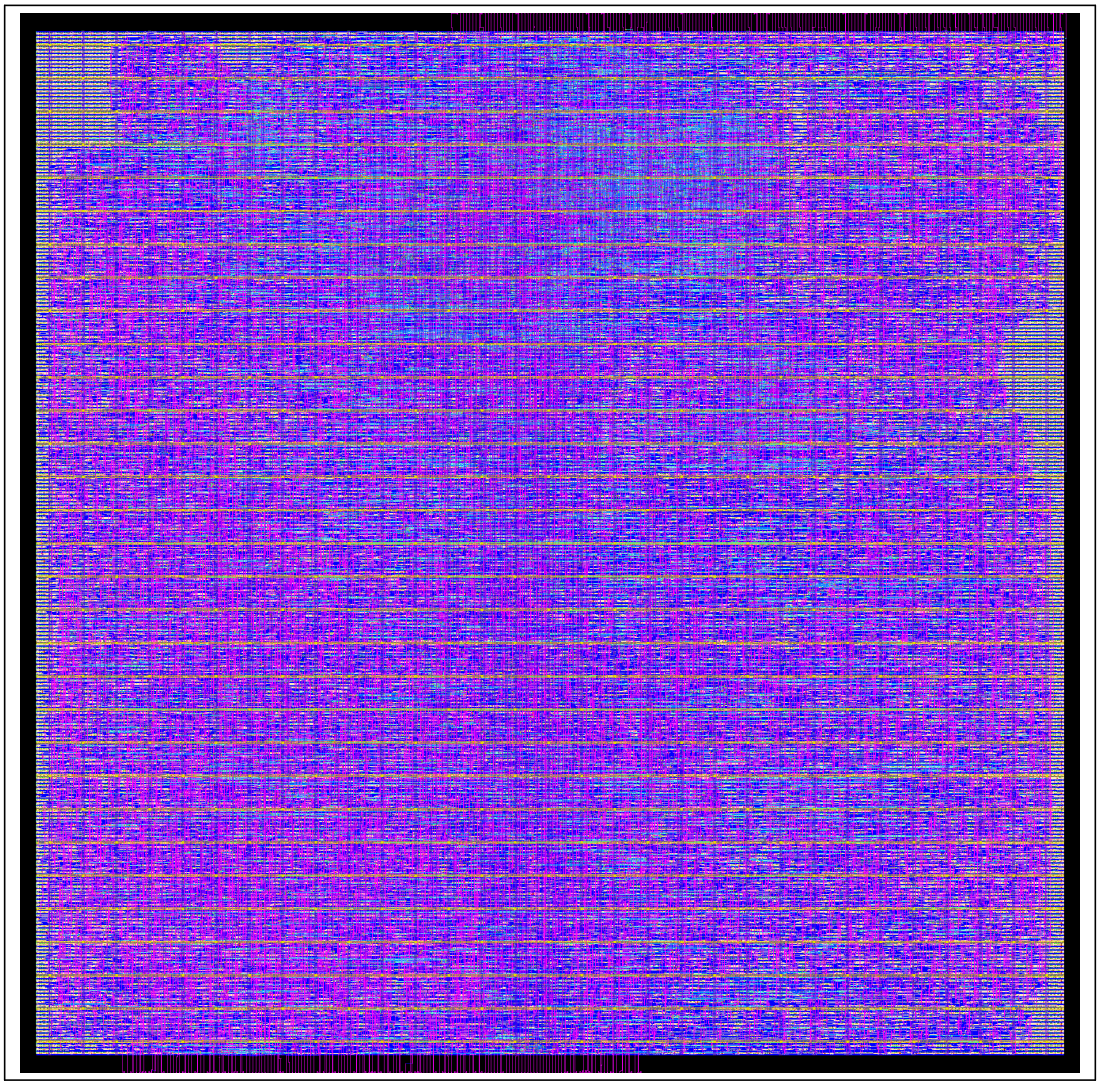


Figure 7: Whole routed chip, all layers (798 um x 798 um)

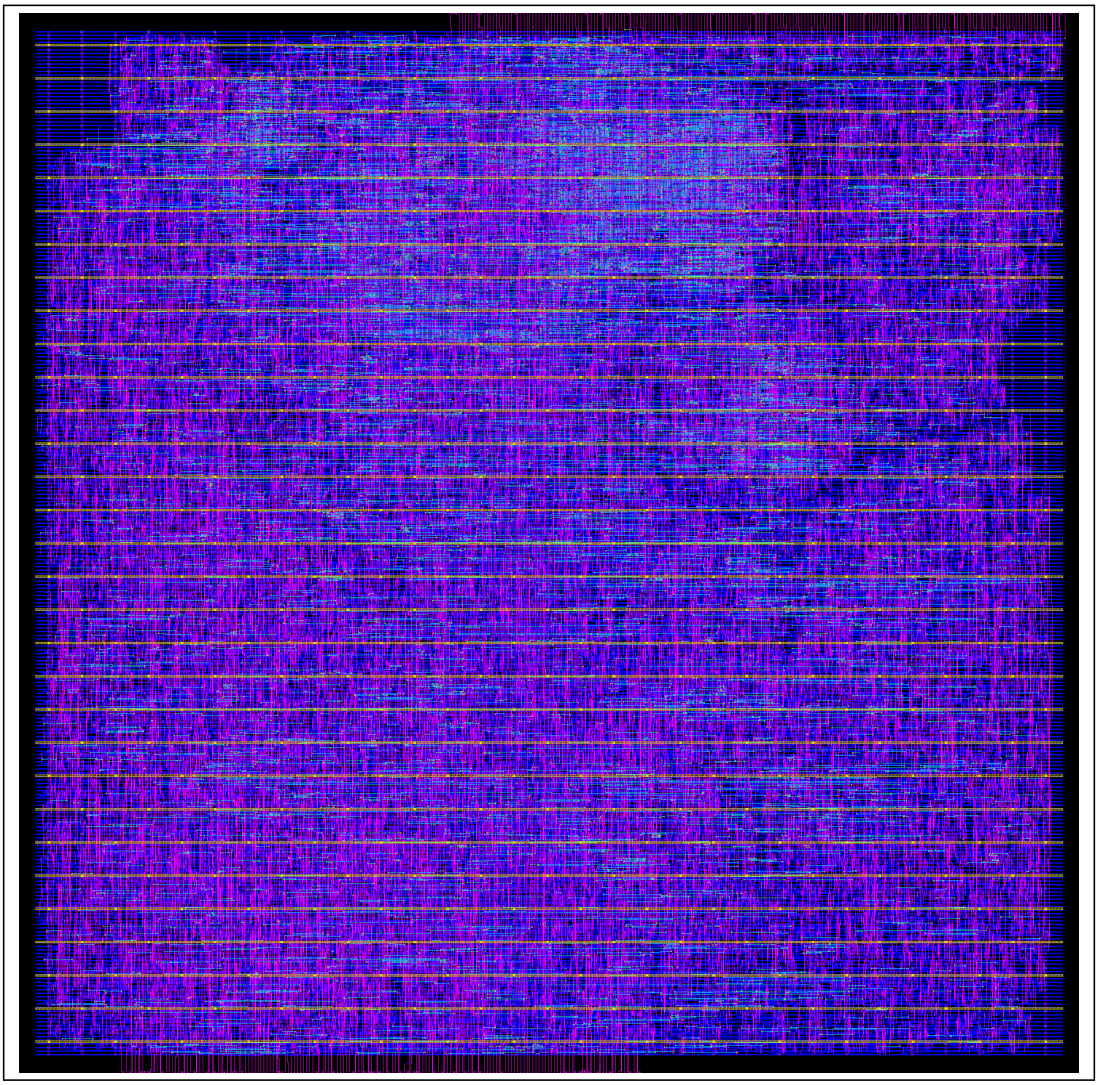


Figure 8: Metal layers only (met1-met5 and vias): the wiring of the chip

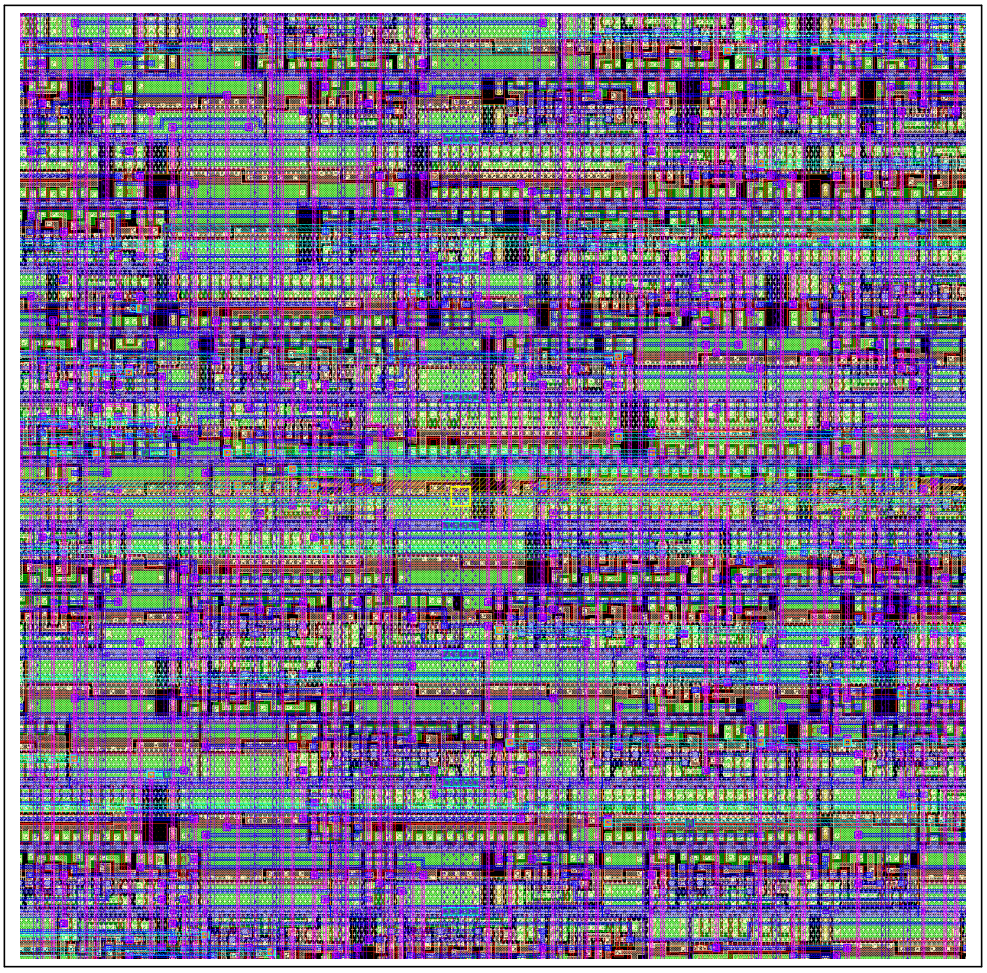


Figure 9: 40 um x 40 um window at the centre: rows of cells with routed wires on top

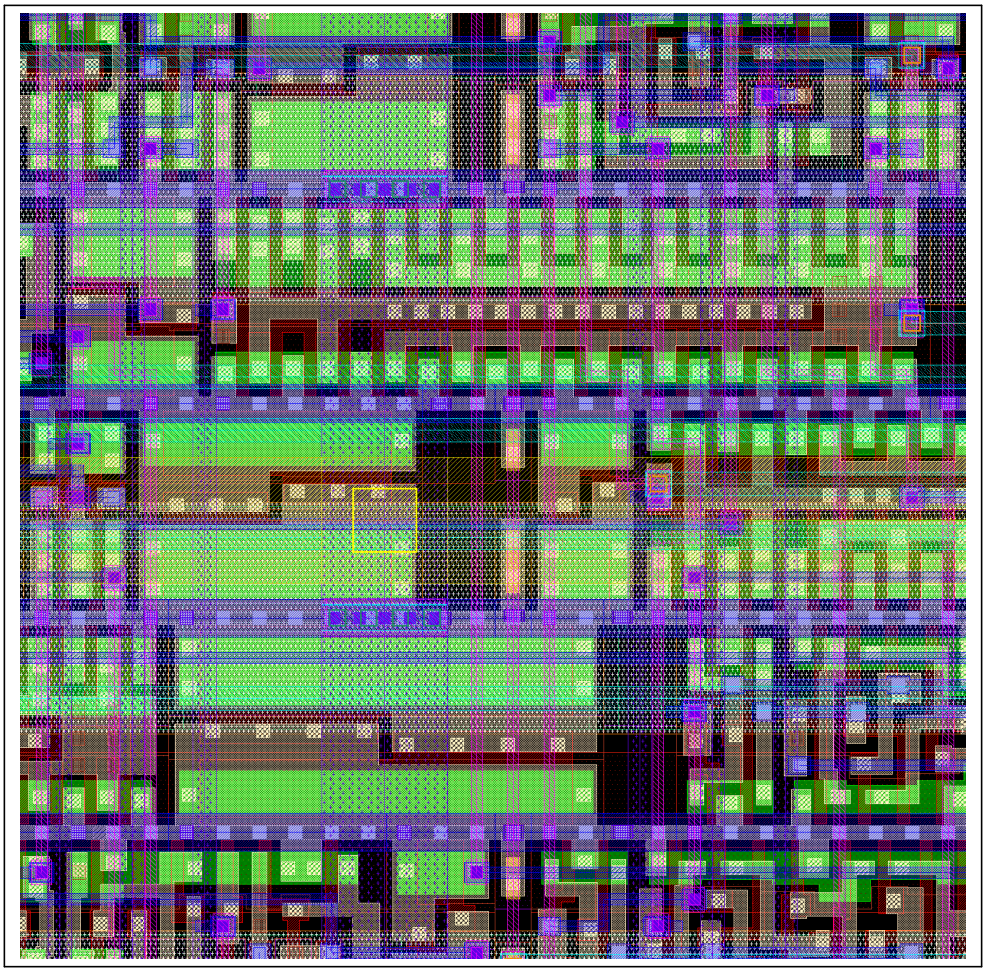


Figure 10: 12 um x 12 um close-up: transistor diffusion (green), polysilicon gates (red), local interconnect and metal wires (blue/purple)

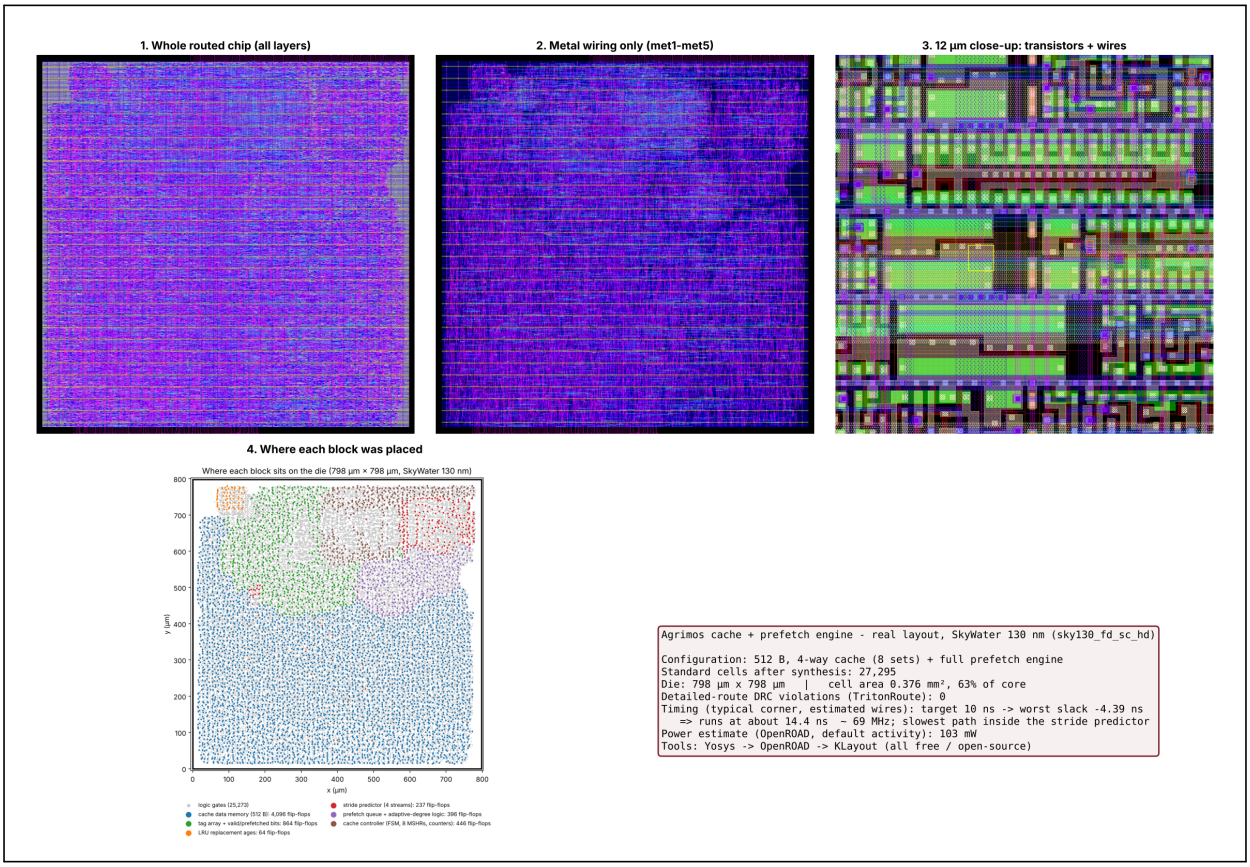


Figure 11: One-page overview of the physical design with its measured numbers

13. Challenges, Fixes and Limitations

13.1 Problems met and how they were solved

Table 17: Problems and fixes during physical design

Problem	Cause	Fix
No prebuilt OpenROAD download allowed in the environment	network allow-list blocked GitHub release files	installed OpenROAD and the PDK from the litex-hub conda channel with the py-rattler installer
Newest OpenROAD package would not install	depends on packages from a channel that was not reachable	used the 2022 build, whose dependencies are all on conda-forge
Yosys check failed: 195 'used but has no driver'	liberty cells were mapped without their port directions	added read_liberty -lib before mapping; check now passes
'global_connect' command not found	older OpenROAD applies global connections inside pdngen	call it only if it exists
4 KB design too big for the free flow	about 39,000 flip-flops of storage	512 B configuration of the same RTL, re-verified by simulation
Full-chip picture showed only cell outlines	marker layers drawn on top of everything	hid area-id/boundary layers; added a metal-only view

13.2 Limitations (stated honestly)

- **512 B, not 4 KB.** A production cache would use an SRAM macro (for example an OpenRAM-generated SKY130 SRAM) instead of flip-flops for the data array; that was not attempted.
- **Timing not met at 100 MHz**; about 69 MHz at the typical corner. Slow and fast corners were not analysed.

- **Parasitics estimated, not extracted.** Timing and power use wire RC estimated from routing, not a SPEF extraction.
- **Partial sign-off only.** Routing DRC is clean in TritonRoute, but Magic/KLayout full-deck DRC, LVS and antenna checks were not run, and no post-layout gate-level simulation was done.
- **Low-power-flow cells.** Synthesis used 336 'lpflow' isolation cells as ordinary logic gates. They are valid cells, but production flows usually exclude them; a future run should mark them don't-use in Yosys as well.
- **Power uses default activity**, not switching activity from the workload traces.
- **Not a tape-out.** No I/O pads, seal ring or chip-level integration were added.

13.3 Future work

- Pipeline or speed up the stride subtraction and re-run to reach 100 MHz.
- Replace the data and tag arrays with SRAM macros and lay out the full 4 KB cache.
- Add multi-corner STA, SPEF extraction, Magic DRC, Netgen LVS and post-layout gate-level simulation.
- Feed VCD activity from the AI traces into report_power for workload-based power numbers.

14. Conclusion

The Agrimos cache and prefetch engine was taken from Verilog RTL to a routed SkyWater 130 nm layout using only free, open-source tools. The 512 B configuration synthesises to 27,295 cells, fits a 798 um square die, and routes with zero DRC violations. It runs at about 69 MHz at the typical corner and uses an estimated 103 mW. The prefetch intelligence costs only about 10 % of the flip-flops, and the placement tool naturally grouped every block into its own region.

The layout also shows clearly what to improve next: the stride subtractor sets the clock speed, and the data array belongs in an SRAM macro. Both are standard engineering steps, and both are documented in the repository so the flow can be re-run.

References

[1] T. Ajayi et al., "Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project," Proc. DAC, 2019.

[2] C. Wolf, "Yosys Open SYnthesis Suite," https://yosyshq.net/yosys/.

[3] R. Brayton and A. Mishchenko, "ABC: An Academic Industrial-Strength Verification Tool," Proc. CAV, 2010.

[4] C.-K. Cheng, A. B. Kahng, I. Kang and L. Wang, "RePlAce: Advancing Solution Quality and Routability Validation in Global Placement," IEEE TCAD, 2019.

[5] A. B. Kahng, L. Wang and B. Xu, "TritonRoute: An Initial Detailed Router for Advanced VLSI Technologies," Proc. ICCAD, 2018.

[6] SkyWater Technology and Google, "SKY130 Open Source PDK," https://github.com/google/skywater-pdk.

[7] M. Koefferlein, "KLayout - Layout Viewer and Editor," https://www.klayout.de.

[8] N. Weste and D. Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed., Addison-Wesley, 2011.

[9] Project repository (RTL, flow scripts, logs):
https://github.com/Kishore-Kotapati/micron_cache_prefetch

Appendix A - Files of the Physical Design

Table 18: Physical-design files in the repository

File	Content
physical/layout_top.v	layout wrapper, cache instantiated with SETS = 8
physical/tb_layout_cfg.v, sim_512B_results.txt	testbench variant and its 18 simulation results
physical/synth_sky130.tcl, layout_top.sdc	Yosys script and timing constraints
physical/flow.tcl, run_layout.sh	OpenROAD flow and one-command runner
physical/render_layout.py, block_map.py, make_overview.py	GDS stream-out, pictures, block map, overview
physical/out/chip.gds.gz	final layout (GDSII, compressed)
physical/out/layout_top.netlist.v, layout_top.final.v	synthesised and final (post-route) netlists
physical/out/openroad.log, final_reports.txt, synth_stat.txt, yosys.log	full logs and reports used in this document
docs/make_layout_report.py	the script that builds this PDF from those logs

Appendix B - Commands to Reproduce

```
# 1. functional check of the 512 B configuration
iverilog -g2005 -o simulation/build/sim8.vvp physical/tb_layout_cfg.v rtl/*.v
vvp -n simulation/build/sim8.vvp +trace=traces/cnn.trace +mode=3 +adaptive=1 +degree=2 +inslru=1

# 2. layout (needs yosys, openroad, pip install klayout, sky130A PDK)
export PDK_ROOT_SKY130=/path/to/share/pdk/sky130A
export OPENROAD=/path/to/openroad
bash physical/run_layout.sh

# 3. look at the chip
gunzip -k physical/out/chip.gds.gz && klayout physical/out/chip.gds

# 4. rebuild this report
python3 docs/make_layout_report.py
```